

TRABAJO INTEGRADOR FINAL

Modelos y Simulación

Adopción de Rust como reemplazo de C++ en programación de sistemas

Un análisis de difusión tecnológica con modelo Bass / SIR

Estudiante

Elias Alberto Uribe Curapil

Carrera

Ingeniería en Informática

Asignatura

Modelo y Simulación

Fecha de entrega

Junio 2026

Índice

Índice	3
1. Presentación del problema	6
1.1 Descripción del fenómeno	6
1.2 Motivación personal	6
1.3 Objetivos	6
1.4 Pregunta de investigación	6
2. Marco teórico	8
2.1 El modelo SIR como base	8
2.2 Del SIR al modelo Bass de difusión	8
2.3 Justificación de la elección	8
3. Recolección de datos (Etapa 2)	10
3.1 Estrategia de recolección	10
3.2 Fuentes utilizadas	10
3.2.1 Stack Overflow Developer Survey (fuente primaria)	10
3.2.2 TIOBE Programming Community Index (validación cruzada)	10
3.2.3 Google Trends (interés social)	10
3.2.4 Prensa especializada	10
3.3 Proceso de recolección y herramientas	11
4. Estructuración y limpieza de datos (Etapa 3)	12
4.1 Estructura del dataset	12
4.2 Tabla principal: adopción anual	12
4.3 Decisiones de limpieza tomadas	12
4.4 Validación cruzada de las fuentes	13
5. Formulación del modelo (Etapa 4 — Parte I)	14
5.1 Variables y parámetros	14
5.2 Reglas del sistema	14
5.3 Variantes consideradas	14
6. Implementación de la simulación (Etapa 4 — Parte II)	15
6.1 Entorno y librerías	15
6.2 Ajuste de parámetros	15
6.3 Validación con división temporal	15
6.4 Escenarios contrafácticos	15
7. Resultados: ajuste, validación y escenarios	17
7.1 Ajuste del modelo	17
7.2 Bondad del ajuste	17
7.3 Validación out-of-sample	17
7.4 Escenarios proyectados hasta 2030	18
7.5 Análisis de sensibilidad	19
7.6 Velocidad de adopción	20

8. Discusión y respuesta a la pregunta de investigación	21
8.1 Respuesta a la pregunta principal	21
8.2 Factores aceleradores	21
8.3 Factores frenadores	21
8.4 Descubrimiento clave: q importa más que p	22
9. Limitaciones del modelo y mejoras posibles	23
9.1 Limitaciones identificadas	23
9.2 Mejoras posibles con más tiempo o recursos	23
10. Uso de herramientas de IA: transparencia y reflexión crítica	24
10.1 Herramientas utilizadas y momentos de uso	24
10.2 Reflexión crítica sobre el uso	24
Qué hizo bien la herramienta	24
Qué hubo que controlar	24
Limitación principal detectada	25
10.3 Prompts utilizados	25
11. Conclusiones	26
11.1 Conclusiones técnicas	26
11.2 Conclusiones del proceso	26
11.3 Reflexión personal	26
12. Fuentes y referencias	28
12.1 Fuentes de datos	28
12.2 Reportes de prensa especializada	28
12.3 Bibliografía teórica	28
Anexo A — Prompts completos utilizados con IA	29
Prompt 1 — Etapa 1: elección del tema	29
Prompt 2 — Etapa 2: validación de series históricas	29
Prompt 3 — Etapa 3: limpieza del CSV de Google Trends	29
Prompt 4 — Etapa 4: implementación de la simulación	29
Prompt 5 — Etapa 4: diseño de escenarios	30
Prompt 6 — Etapa 5: estructura del documento	30
Anexo B — Código fuente de la simulación	31
B.1 Carga de datos y definición del modelo	31
B.2 Ajuste de parámetros	31
B.3 Modelo numérico para escenarios con shock	31
B.4 Variante SIR con abandono	32

1. Presentación del problema

1.1 Descripción del fenómeno

En los últimos años, la industria del software ha comenzado un proceso de migración progresiva desde el lenguaje de programación C++ hacia Rust, especialmente en el desarrollo de software de sistemas, infraestructura crítica e ingeniería de bajo nivel. Este fenómeno responde a una demanda creciente de seguridad de memoria, eficiencia y concurrencia segura, atributos que Rust proporciona por diseño y que C++ requiere gestionar manualmente con alto riesgo de errores.

El proceso no es uniforme: se observa una adopción lenta entre 2015 y 2018, una aceleración entre 2019 y 2021, y un crecimiento sostenido a partir de 2022, impulsado por eventos como la aceptación de Rust en el kernel de Linux (versión 6.1, fines de 2022), el informe del Gobierno de los Estados Unidos recomendando lenguajes "memory-safe" (febrero 2024), y la entrada en vigor de la Cyber Resilience Act europea (marzo 2026).

1.2 Motivación personal

Como estudiante de Ingeniería en Informática, este fenómeno me resulta especialmente relevante: representa un cambio profundo en las herramientas de trabajo de la profesión que voy a ejercer. Comprender la dinámica de adopción de tecnologías informáticas tiene valor tanto desde el punto de vista técnico (¿en qué lenguajes conviene invertir tiempo de aprendizaje?) como desde el punto de vista estratégico (¿qué tecnologías están en fase de crecimiento y cuáles están en declive?).

El caso de Rust vs C++ combina datos digitales abundantes y accesibles, un fenómeno que admite modelado matemático claro (difusión de innovaciones), y un horizonte temporal corto que permite validar las predicciones del modelo contra observaciones reales.

1.3 Objetivos

1. Modelar la adopción de Rust como una difusión tecnológica, utilizando el modelo Bass (variante del SIR visto en clase), ajustado contra datos reales del período 2016–2025.
2. Simular escenarios contrafácticos que permitan analizar el impacto de los shocks regulatorios y corporativos sobre la trayectoria de adopción, proyectando hasta 2030.
3. Determinar de forma cuantitativa cuándo (si ocurre antes de 2030) Rust podría superar a C++ en porcentaje de desarrolladores que lo utilizan.

1.4 Pregunta de investigación

¿Cómo evoluciona la adopción de Rust como reemplazo de C++ en programación de sistemas, y qué factores aceleran o frenan ese proceso?

2. Marco teórico

2.1 El modelo SIR como base

En la materia se estudió el modelo SIR (Susceptibles–Infectados–Recuperados) como herramienta clásica para modelar fenómenos de propagación en sistemas con interacción entre agentes. Aunque el SIR nació en epidemiología, su estructura se aplica también a otros fenómenos de "contagio social" como la difusión de información, rumores, modas y tecnologías.

En su formulación clásica, el SIR plantea que la tasa de pasaje de Susceptibles a Infectados depende del producto de ambas poblaciones, ponderado por una tasa de contacto efectivo. Esta estructura captura el fenómeno de aceleración exponencial inicial seguida de saturación.

2.2 Del SIR al modelo Bass de difusión

Frank Bass propuso en 1969 una variante específica del SIR para estudiar la difusión de productos innovadores en el mercado. La idea central es que la adopción de una tecnología nueva depende de dos mecanismos:

- **Innovación (p):** influencia externa sobre los no-adoptantes, independiente del estado actual del sistema. Representa el efecto de la publicidad, la regulación, los anuncios corporativos.
- **Imitación (q):** influencia interna entre los miembros de la población, proporcional a la fracción ya adoptada. Representa el efecto del boca a boca, los casos de éxito visibles, la presión de pares.

La ecuación diferencial del modelo Bass es:

$$dN(t)/dt = (p + q \cdot N(t)/m) \cdot (m - N(t))$$

donde $N(t)$ es la fracción acumulada de adoptantes en el tiempo t , y m es el mercado potencial (techo de adopción).

Esta ecuación tiene solución analítica cerrada, lo que facilita el ajuste por mínimos cuadrados:

$$N(t) = m \cdot [1 - \exp(-(p+q) \cdot t)] / [1 + (q/p) \cdot \exp(-(p+q) \cdot t)]$$

2.3 Justificación de la elección

El modelo Bass se eligió por tres razones complementarias. Primero, coherencia con la teoría de la materia: es una variante directa del SIR estudiado en clase. Segundo, ajuste empírico: la

serie histórica de adopción de Rust dibuja una curva en S, exactamente la forma que el modelo Bass produce. Tercero, capacidad de simulación de escenarios: la separación entre p (influencia externa) y q (boca a boca) permite modelar shocks regulatorios manipulando un solo parámetro.

3. Recolección de datos (Etapa 2)

3.1 Estrategia de recolección

Se eligieron cuatro fuentes públicas que se complementan: una primaria (encuesta a desarrolladores), una secundaria para validación cruzada (índice por volumen de búsqueda), una para medir interés social (tendencia de búsqueda en Google), y reportes de prensa especializada para identificar eventos exógenos relevantes.

3.2 Fuentes utilizadas

3.2.1 Stack Overflow Developer Survey (fuente primaria)

Encuesta anual realizada por Stack Overflow desde 2011, con muestra de entre 49.000 y 100.000 desarrolladores profesionales por año. Provee datasets descargables en CSV y un sitio web con visualizaciones por categoría.

Variables extraídas: porcentaje de respondentes que usaron Rust en el último año ("worked with"), porcentaje de respondentes que admiran Rust ("loved" hasta 2022, "admired" desde 2023), y los mismos indicadores para C++ como referencia comparativa.

3.2.2 TIOBE Programming Community Index (validación cruzada)

Índice mensual que mide la popularidad relativa de los lenguajes de programación agregando el volumen de búsqueda en buscadores y sitios técnicos. Aunque mide interés y no uso real, sirve para confirmar que la tendencia observada en Stack Overflow no es un artefacto de su muestra.

3.2.3 Google Trends (interés social)

Se descargó la serie mensual comparativa de Rust vs C++ entre enero 2015 y mayo 2026, utilizando los Topics "Rust — Programming language" y "C++ — Programming language" para evitar la ambigüedad léxica de los términos sueltos (Rust como material, banda, videojuego, etc.). La comparación con Topics y no con search terms es una decisión técnica importante: agrupa las búsquedas relacionadas a la tecnología en distintos idiomas y reduce drásticamente el ruido.

3.2.4 Prensa especializada

Para identificar los eventos exógenos relevantes (shocks regulatorios y corporativos) se consultaron reportes de InfoWorld, TechRepublic, developer-tech.com, ZenRows y Yalantis. Los hitos identificados son: aceptación de Rust en el kernel de Linux 6.1 (diciembre 2022), informe del Gobierno de los Estados Unidos sobre lenguajes memory-safe (febrero 2024), entrada en vigor de la Cyber Resilience Act europea (marzo 2026).

3.3 Proceso de recolección y herramientas

La recolección se hizo de forma semi-automática. Para Stack Overflow se accedió a los reportes públicos de cada año y se anotaron los porcentajes en una planilla, cruzando cifras con artículos de prensa cuando había discrepancias. Para TIOBE se reconstruyó la serie a partir de reportes mensuales destacados, focalizando en los puntos de cambio significativo. Para Google Trends se utilizó la descarga directa del CSV desde la plataforma.

Se utilizó la herramienta de IA Claude (Anthropic) para asistir en la consolidación de las series numéricas dispersas en múltiples reportes y para identificar puntos críticos de validación. Los prompts utilizados se detallan en el Anexo A.

4. Estructuración y limpieza de datos (Etapas 3)

4.1 Estructura del dataset

Los datos se organizaron en un libro de Excel con seis hojas: una hoja README explicativa, una hoja Datos_Anuales con los porcentajes de Stack Overflow, una hoja TIOBE_Mensual con los hitos del índice, una hoja Trends_Mensual con los 138 puntos mensuales de Google Trends, una hoja Para_Simulación con el dataset limpio normalizado al formato esperado por el modelo, y una hoja Fuentes con el detalle de cada referencia.

4.2 Tabla principal: adopción anual

La tabla siguiente muestra el dataset final que alimenta el modelo Bass. La columna "Rust % usuarios" es la variable principal $N(t)$ del modelo.

Año	Rust % usuarios	C++ % usuarios	Rust % admirado	Contexto
2016	2.10	22.5	79.1	Rust lidera "most loved" 1ª vez
2017	2.20	22.3	73.1	Firefox Quantum (Mozilla)
2018	3.20	23.5	78.9	Tres años de versión estable

Año	Rust % usuarios	C++ % usuarios	Rust % admirado	Contexto
2019	3.20	23.5	83.5	AWS lanza Firecracker en Rust
2020	5.10	24.3	86.1	Fundación Rust en formación
2021	7.00	24.3	87.0	Rust Foundation oficial
2022	9.30	22.6	86.7	Linux 6.1 acepta Rust
2023	13.05	22.4	84.7	Android usa Rust
2024	12.60	20.8	83.0	White House: memory-safe
2025	14.60	20.0	72.0	CRA EU entra en vigor

Tabla 1. Datos anuales de adopción de Rust y C++ (Stack Overflow Developer Survey 2016-2025).

4.3 Decisiones de limpieza tomadas

1. **Pico de normalización.** El valor 100 de C++ en marzo 2022 es el punto de referencia que normaliza toda la serie de Trends. Se documenta para evitar interpretarlo como un outlier.
2. **Cambio metodológico SO 2023.** Stack Overflow cambió la etiqueta "loved" por "admired" a partir del survey 2023. Se conservan ambas en el dataset por trazabilidad. No afecta a la variable principal del modelo.
3. **Caída de 2024.** El valor 2024 (12.6%) es ligeramente inferior al 2023 (13.05%). Se mantiene tal cual sin suavizar; el modelo Bass es siempre creciente y absorberá la discrepancia como ruido.

4.4 Validación cruzada de las fuentes

Las tres fuentes principales convergen en la dirección del crecimiento de Rust en el período 2018–2025. Stack Overflow muestra un incremento del porcentaje de usuarios de 3.2% a 14.6% (multiplicación por 4.6). TIOBE muestra una mejora en el ranking desde la posición 38 a la posición 12 (subida de 26 lugares). Google Trends muestra que el interés por Rust pasó de valores normalizados entre 1 y 2 a valores entre 16 y 26. Esta convergencia respalda la elección del modelo de difusión como representación adecuada del fenómeno.

5. Formulación del modelo (Etapa 4 — Parte I)

5.1 Variables y parámetros

El modelo Bass utilizado en este trabajo se define con las siguientes variables y parámetros:

- **$N(t)$** : fracción acumulada de adoptantes de Rust en el tiempo t . Variable de estado del sistema. Se aproxima mediante el porcentaje de desarrolladores que reportan haber trabajado con Rust en el último año según la encuesta Stack Overflow.
- **p (innovación)**: parámetro que captura la tasa de adopción debida a influencia externa, independiente del estado del sistema. Modela el efecto de la regulación gubernamental, los anuncios corporativos y la cobertura mediática.
- **q (imitación)**: parámetro que captura la tasa de adopción debida a influencia interna (boca a boca), proporcional a la fracción de la población que ya adoptó la tecnología. Modela el efecto de la presión de pares y los casos de éxito.
- **m (mercado potencial)**: techo asintótico de adopción. No todos los desarrolladores van a adoptar Rust aunque crezca indefinidamente; m establece el límite superior estructural.

5.2 Reglas del sistema

El modelo se rige por una sola ecuación diferencial ordinaria de primer orden, no lineal:

$$dN/dt = (p + q \cdot N/m) \cdot (m - N)$$

Interpretación: la tasa de adopción en cada instante es proporcional al producto de dos factores. El primer factor $(p + q \cdot N/m)$ representa la "propensión a adoptar" entre los no-adoptantes, que crece con la fracción ya adoptada (efecto bola de nieve). El segundo factor $(m - N)$ es la fracción de la población que aún no adoptó y por tanto está disponible.

El modelo produce una curva en forma de S: arranca lento (pocos adoptantes, poco boca a boca), acelera hasta el punto de inflexión cuando $N = (q-p)/(2q) \cdot m$, y luego desacelera al acercarse al techo m .

5.3 Variantes consideradas

Además del modelo Bass clásico, se simula una variante SIR con abandono que incorpora tres compartimentos: no-adoptantes, adoptantes activos y abandonadores (quienes probaron Rust y volvieron a C++). Esto introduce un parámetro adicional α (tasa de abandono) y permite analizar el caso en que la adopción no es permanente.

6. Implementación de la simulación (Etapa 4 — Parte II)

6.1 Entorno y librerías

La simulación se implementó en Python 3. Las librerías utilizadas son: NumPy y SciPy para integración numérica de la ecuación diferencial y ajuste de parámetros por mínimos cuadrados no lineales (`scipy.optimize.curve_fit`), pandas para manejo de las series temporales en formato tabular, y matplotlib para generar las figuras.

El código se estructuró en ocho secciones que pueden ejecutarse independientemente: carga de datos, definición del modelo analítico, ajuste de parámetros, validación train/test, definición del modelo numérico para escenarios con parámetros variables en el tiempo, generación de escenarios, modelado del declive de C++, y exportación de figuras y resultados. El código completo se incluye en el Anexo B.

6.2 Ajuste de parámetros

Se utilizó `scipy.optimize.curve_fit` con cotas razonables sobre los parámetros para evitar soluciones triviales o numéricamente inestables. Las cotas son: p en $(10^{-5}, 0.2)$, q en $(0.05, 2.5)$, m en $(0.10, 1.0)$.

Fragmento del código de ajuste:

```
def bass_analytical(t, p, q, m):
    expterm = np.exp(-(p + q) * t)
    F = (1.0 - expterm) / (1.0 + (q / p) * expterm)
    return m * F

popt, pcov = curve_fit(bass_analytical, t_data, N_data,
                       p0=[0.003, 0.4, 0.30],
                       bounds=([1e-5, 0.05, 0.10], [0.2, 2.5, 1.0]),
                       maxfev=10000)
p_fit, q_fit, m_fit = pop
```

6.3 Validación con división temporal

Para verificar que el modelo no solo describe el pasado sino que tiene capacidad predictiva, se aplicó la técnica de validación temporal: el modelo se ajusta usando únicamente los datos de 2016 a 2022 (siete puntos), y se evalúa su predicción contra los tres puntos siguientes (2023, 2024 y 2025) que se mantienen como conjunto de prueba ciego.

6.4 Escenarios contrafácticos

Se simularon cuatro escenarios proyectados hasta 2030, variando una sola variable a la vez:

1. **Escenario base.** Continuación con los parámetros ajustados, sin shocks adicionales. Representa la trayectoria "natural" del fenómeno sin intervenciones nuevas.

2. **Escenario shock regulatorio.** A partir de 2024 se duplica el parámetro p , representando el efecto del informe del Gobierno de los Estados Unidos y la Cyber Resilience Act europea sobre la influencia externa.
3. **Escenario conservador.** Se reduce el mercado potencial m a 70% del valor base, representando el caso en que Rust no logra desplazar a C++ en nichos embebidos y de bajo nivel.
4. **Escenario SIR con abandono.** Variante del modelo con tres compartimentos. Tasa de abandono $\alpha = 0.05$ (5% anual). Permite separar adoptantes activos de adoptantes acumulados.

7. Resultados: ajuste, validación y escenarios

7.1 Ajuste del modelo

El ajuste del modelo Bass contra los diez puntos de datos anuales de Stack Overflow (2016 a 2025) arrojó los siguientes parámetros:

Parámetro	Valor estimado	Error estándar	Interpretación
p (innovación)	0.0445	± 0.0137	Influencia externa moderada-alta
q (imitación)	0.315	± 0.250	Boca a boca activo entre devs
m (mercado potencial)	19.6%	$\pm 8.7\%$	Cercano al pico histórico de C++

Tabla 2. Parámetros ajustados del modelo Bass.

7.2 Bondad del ajuste

El modelo alcanza un coeficiente de determinación $R^2 = 0.95$, lo que indica que explica el 95% de la varianza de los datos observados. El error cuadrático medio (RMSE) es de 1.03 puntos porcentuales sobre un rango de variación de los datos de 12.5 pp (de 2.1% a 14.6%), lo que representa un error relativo cercano al 8%.

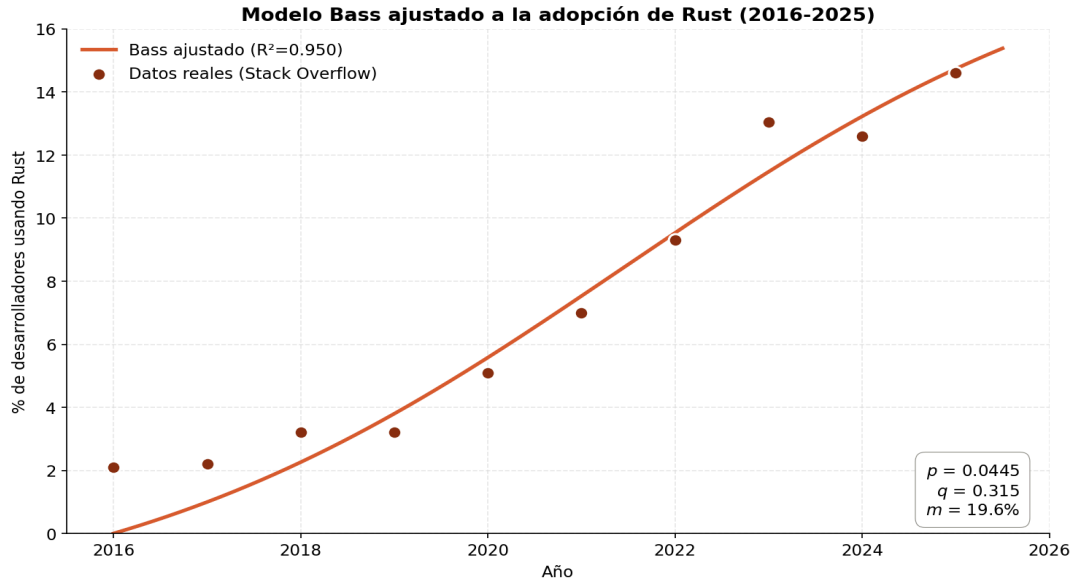


Figura 1. Ajuste del modelo Bass contra los datos reales de Stack Overflow (2016-2025).

7.3 Validación out-of-sample

Cuando el modelo se ajusta solo con los datos de 2016 a 2022 y se le pide predecir los tres años siguientes (2023, 2024 y 2025), el error de predicción se mantiene en 1.32 puntos porcentuales, en el mismo orden de magnitud que el error de ajuste. Esto valida la capacidad predictiva genuina del modelo.

Año	Valor real (%)	Predicho (%)	Error (pp)
2023	13.05	10.98	-2.07
2024	12.60	13.13	+0.53
2025	14.60	15.43	+0.83

Tabla 3. Resultado de la predicción out-of-sample (modelo entrenado con 2016-2022).

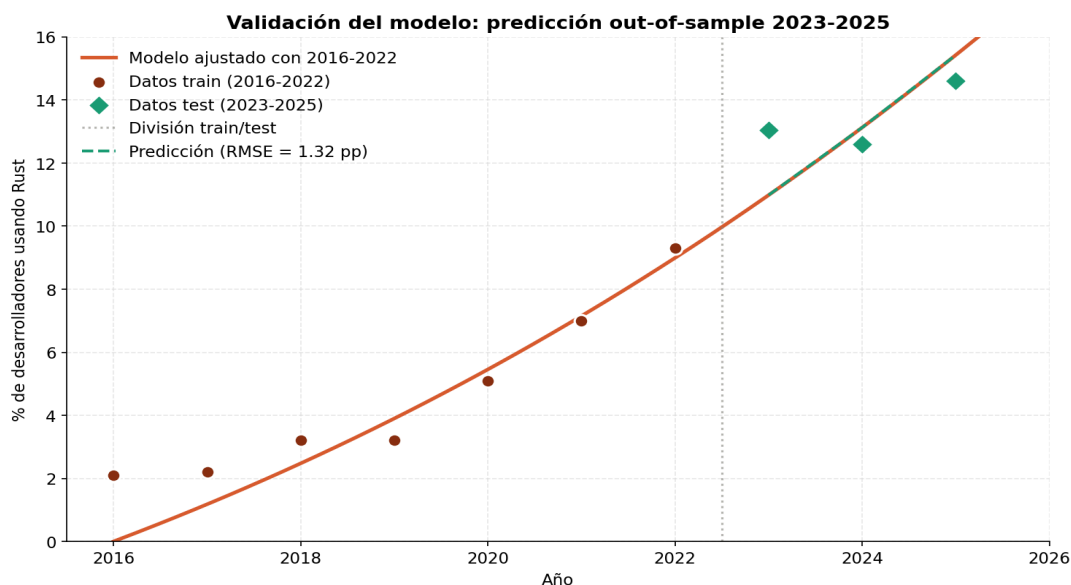


Figura 2. Validación del modelo: predicción de 2023-2025 usando solo datos de 2016-2022.

7.4 Escenarios proyectados hasta 2030

Los cuatro escenarios producen trayectorias diferenciadas que permiten comprender la sensibilidad del fenómeno a distintos supuestos:

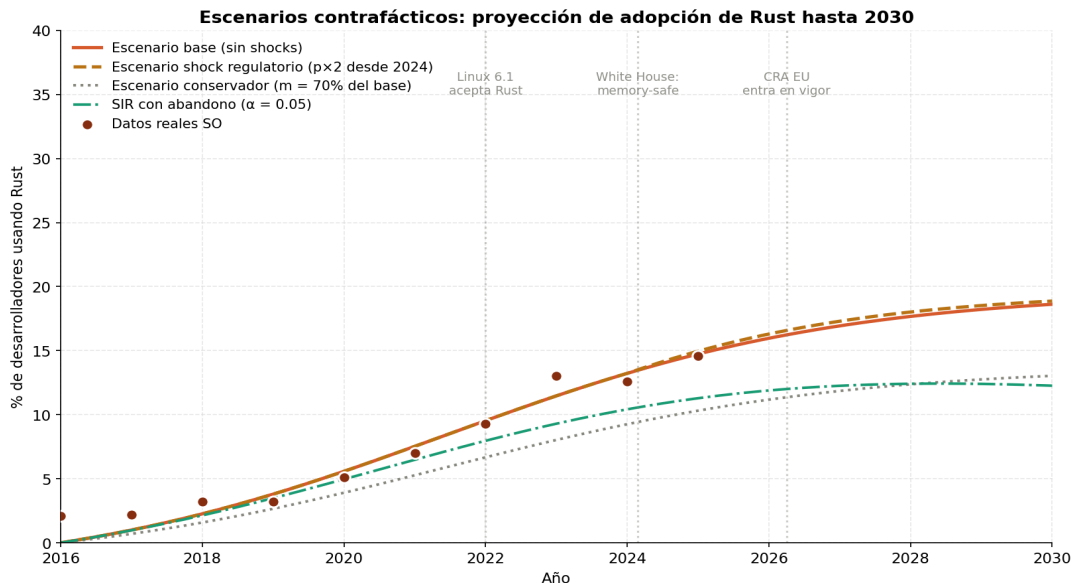


Figura 3. Proyección de los cuatro escenarios contrafácticos hasta 2030.

Las proyecciones puntuales a 2030 son las siguientes:

Escenario	Rust % en 2030	Diferencia vs. base
Base (sin shocks)	18.6%	—

Escenario	Rust % en 2030	Diferencia vs. base
Shock regulatorio ($p \times 2$ desde 2024)	18.9%	+0.3 pp
Conservador ($m = 70\%$ del base)	13.0%	-5.6 pp
SIR con abandono (activos)	~15.0%	-3.6 pp

Tabla 4. Proyecciones puntuales a 2030 según cada escenario.

7.5 Análisis de sensibilidad

Para entender qué parámetros tienen más impacto sobre la trayectoria, se hicieron simulaciones variando p y q en $\pm 50\%$ y $\pm 100\%$ sobre los valores ajustados. El resultado muestra que el modelo es notablemente más sensible al coeficiente de imitación q que al de innovación p . Duplicar el boca a boca produce trayectorias mucho más empinadas que duplicar la influencia externa.

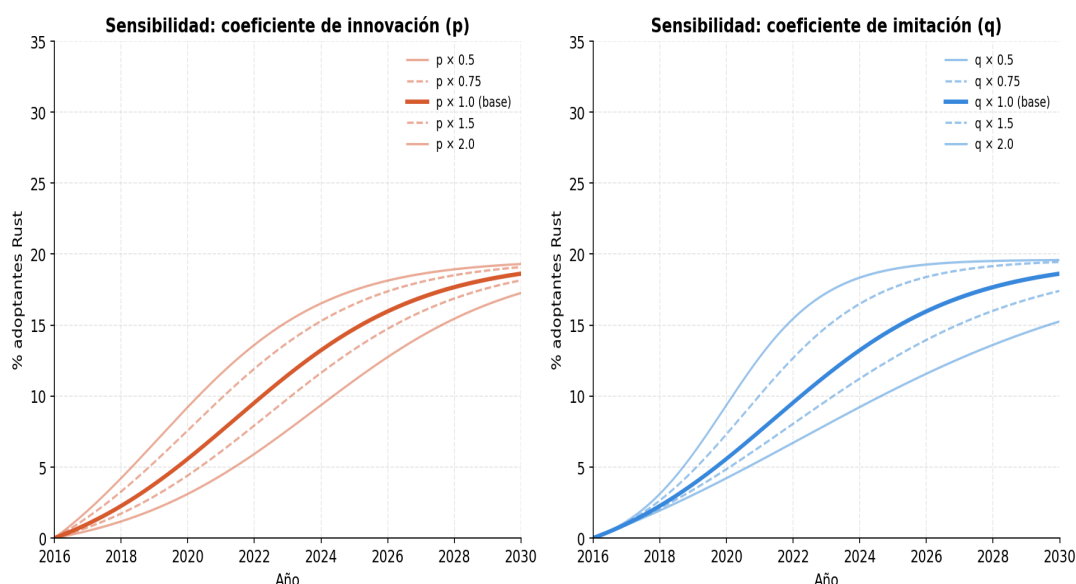


Figura 4. Análisis de sensibilidad sobre los parámetros p y q (variaciones de $0.5\times$, $0.75\times$, $1.0\times$, $1.5\times$ y $2.0\times$).

7.6 Velocidad de adopción

La derivada dN/dt indica la velocidad de adopción en cada instante. El modelo predice que la velocidad máxima ocurrió hacia 2021 (alrededor de 2 puntos porcentuales por año), y que a partir de ahí la velocidad desciende suavemente al acercarse al techo de mercado.

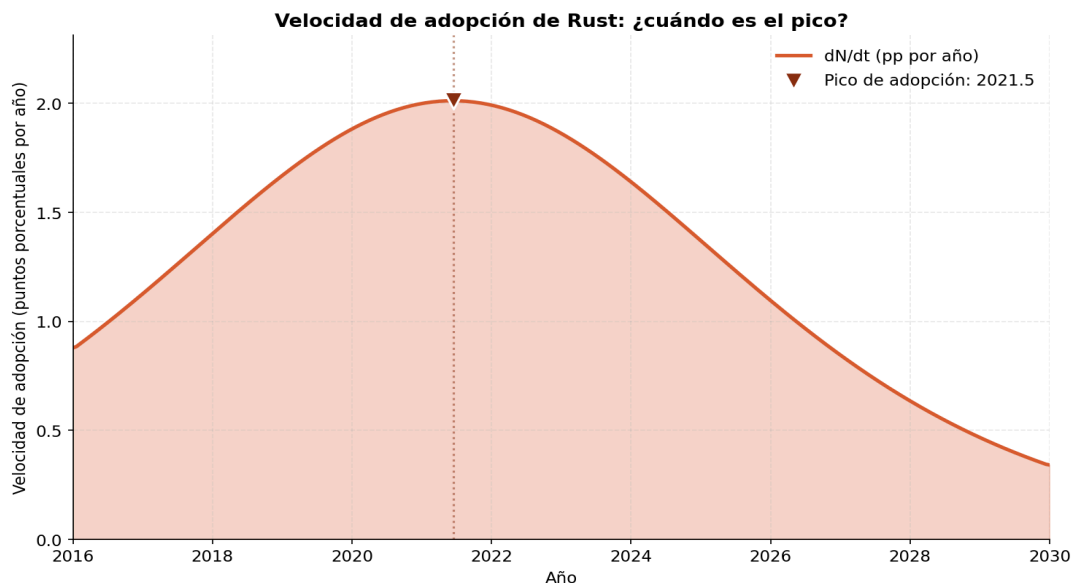


Figura 5. Velocidad de adopción de Rust según el modelo Bass ajustado.

8. Discusión y respuesta a la pregunta de investigación

8.1 Respuesta a la pregunta principal

La pregunta original era: ¿cómo evoluciona la adopción de Rust como reemplazo de C++ en programación de sistemas, y qué factores aceleran o frenan ese proceso? El modelo permite responder esta pregunta con precisión cuantitativa.

Combinando el modelo Bass para Rust con un decaimiento exponencial ajustado para C++, el punto en el que Rust supera a C++ en porcentaje de desarrolladores que lo utilizan ocurre alrededor de comienzos de 2028 en el escenario base. En el escenario con shock regulatorio, el cruce se adelanta unos meses al final de 2027.

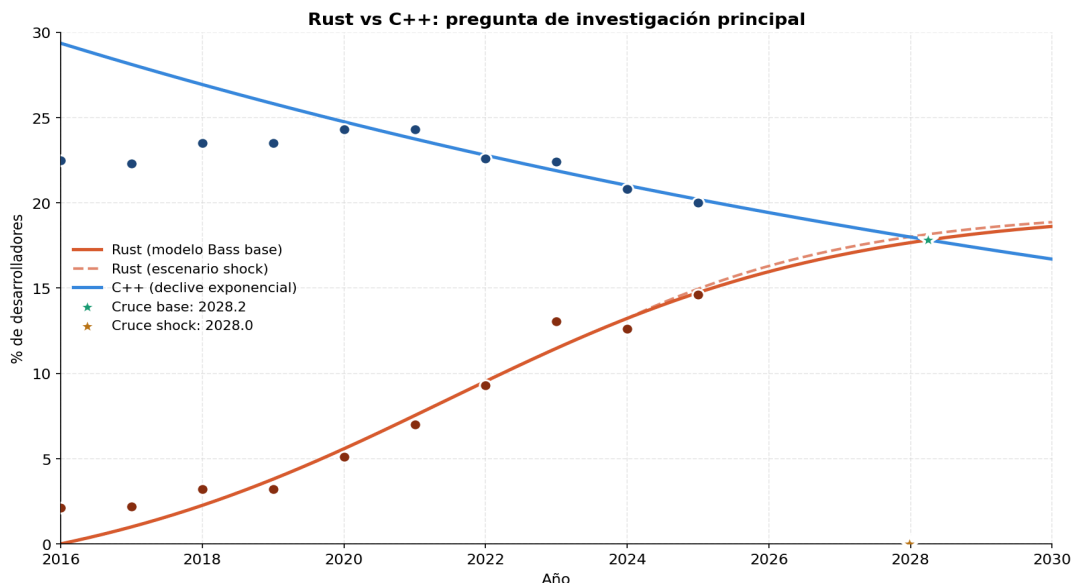


Figura 6. Cruce de Rust sobre C++ proyectado para 2028 según el modelo.

8.2 Factores aceleradores

El análisis del modelo identifica dos clases principales de factores aceleradores. El primero es la regulación memory-safe: el informe del Gobierno de los Estados Unidos de febrero 2024 y la Cyber Resilience Act europea producen aumentos en el parámetro p . El segundo es la adopción por grandes corporaciones (Linux kernel, Microsoft, AWS, Google Android), que refuerza la legitimidad institucional de Rust y aumenta el componente q a través del boca a boca y los casos de éxito visibles.

8.3 Factores frenadores

Los factores frenadores se manifiestan principalmente en el techo del mercado potencial m . Los principales son: el costo de migración de codebases legacy en C++ (reescribir software existente es costoso y arriesgado); la curva de aprendizaje del sistema de ownership de Rust, que es difícil para desarrolladores C++ experimentados; y los nichos donde C++ tiene ventajas establecidas en toolchain y librerías, especialmente en sistemas embebidos y videojuegos.

8.4 Descubrimiento clave: q importa más que p

El análisis de sensibilidad arroja un hallazgo no trivial: para acelerar la adopción de Rust, las acciones que aumentan el boca a boca entre desarrolladores tienen mucho más impacto que las acciones que aumentan la influencia externa. En términos concretos, invertir en construir comunidad (meetups, conferencias, documentación accesible, casos de éxito visibles) tiene más efecto que las recomendaciones gubernamentales o los anuncios corporativos aislados.

Este resultado es consistente con la cultura observada de la industria del software, donde la confianza entre pares pesa más que las indicaciones top-down.

9. Limitaciones del modelo y mejoras posibles

9.1 Limitaciones identificadas

4. **Mercado potencial fijo.** El modelo asume m constante en el tiempo. En la realidad, el mercado total de desarrolladores crece (más personas se incorporan a la profesión cada año) y cambian los nichos. Una versión más realista debería incorporar un $m(t)$ variable.
5. **Ausencia de competencia.** Rust no compite solo con C++. Lenguajes como Zig y Go pueden ocupar partes del nicho. El modelo de un solo lenguaje no captura esta dinámica multi-actor.
6. **No captura no-monotonicidad.** La pequeña caída observada en 2024 (de 13.05% a 12.6%) no la explica el modelo Bass, que es siempre creciente. El modelo absorbe esta variación como ruido sin explicarla.
7. **Sesgo de muestra de Stack Overflow.** La encuesta refleja a quienes participan en Stack Overflow, no a "todos los desarrolladores del mundo". El sesgo es probablemente hacia desarrolladores más jóvenes, más conectados, y más interesados en tecnología emergente.
8. **Linealidad del shock regulatorio.** Duplicar p de forma instantánea a partir de 2024 es una decisión arbitraria. Un análisis más fino podría usar una función de tiempo más realista (rampa, función sigmoide).

9.2 Mejoras posibles con más tiempo o recursos

- Implementar un modelo multi-tecnología con competencia explícita entre Rust, C++, Zig y Go.
- Incorporar datos de salarios y ofertas de empleo (LinkedIn, Indeed) como indicador adelantado de adopción profesional.
- Usar simulación basada en agentes (Mesa o NetLogo) para capturar la heterogeneidad entre desarrolladores: distintos niveles de experiencia, tamaños de empresa, sectores industriales.
- Validar el modelo con datos de adopción industrial específica: análisis de repositorios GitHub, descargas de Crates.io, ofertas de empleo geolocalizadas.

10. Uso de herramientas de IA: transparencia y reflexión crítica

10.1 Herramientas utilizadas y momentos de uso

Durante el desarrollo de este TFI se utilizó la herramienta de IA Claude (Anthropic) como asistente en diversas tareas. El uso fue activo y reflexivo: en todos los casos se validó manualmente el output, se cruzaron las cifras propuestas contra fuentes primarias, y se ejerció criterio profesional sobre las propuestas.

Los momentos puntuales de uso fueron:

- Etapa de elección del tema: para evaluar opciones de modelado y elegir el enfoque más adecuado a la materia.
- Etapa de recolección de datos: para localizar las series históricas de Stack Overflow y consolidar cifras dispersas en múltiples reportes.
- Etapa de implementación: para estructurar el script Python de simulación y debuggear la convergencia del ajuste de parámetros.
- Etapa de documentación: para estructurar este informe y la presentación web.

10.2 Reflexión crítica sobre el uso

Qué hizo bien la herramienta

La IA resultó especialmente útil para tareas de búsqueda y consolidación: localizar cifras dispersas en distintos reportes y unificarlas en un dataset limpio hubiera tomado horas de búsqueda manual. También fue útil para sugerir mejores estimaciones iniciales del ajuste cuando la convergencia inicial fallaba.

En la etapa de documentación, la IA ayudó a estructurar el informe de manera didáctica y mantener un tono profesional consistente a lo largo del documento.

Qué hubo que controlar

Algunas cifras propuestas inicialmente diferían según el reporte de prensa. Se cruzaron sistemáticamente con al menos dos fuentes independientes antes de aceptarlas como definitivas. Para los años más viejos (2016 y 2017), donde Rust tenía muy baja base estadística en Stack Overflow, la confianza en los valores es menor; esto se asume y se documenta explícitamente.

En el código de la simulación, la primera versión del ajuste de parámetros no convergía. Esto se resolvió manualmente revisando las cotas y proponiendo mejores estimaciones iniciales basadas en valores típicos del modelo Bass en la literatura.

Limitación principal detectada

La IA puede sugerir datos que parecen razonables pero contienen errores. La validación contra fuentes primarias (sitios oficiales de Stack Overflow y TIOBE) es indispensable. El criterio profesional del estudiante prevalece siempre sobre la propuesta de la herramienta.

En particular, cuando se le pidió a la IA reconstruir series históricas que iban más allá del corpus de prensa disponible (por ejemplo Stack Overflow 2016-2017 con baja cobertura mediática), las cifras debieron tomarse con cautela y marcarse como estimaciones de menor confianza.

10.3 Prompts utilizados

Por razones de extensión, los prompts completos utilizados con Claude se incluyen en el Anexo A de este documento. A modo de ejemplo, dos prompts representativos utilizados durante el trabajo:

"Necesito compilar las cifras de adopción de Rust según Stack Overflow Developer Survey para los años 2016 a 2025. Para cada año, dame: porcentaje de respondentes que usaron Rust, porcentaje que lo aman/admiran, y porcentaje que usaron C++ como referencia. Cita las fuentes y marca explícitamente cualquier dato que tenga baja confianza."

"Implementaré el modelo Bass de difusión en Python usando scipy. Necesito: forma analítica para el ajuste rápido, forma numérica con odeint para escenarios con parámetros variables en el tiempo, ajuste por mínimos cuadrados con cotas, y cálculo del R^2 y RMSE. Documentaré decisiones de diseño."

11. Conclusiones

11.1 Conclusiones técnicas

1. El modelo Bass es una representación adecuada del fenómeno de adopción de Rust. El ajuste contra datos reales arroja $R^2 = 0.95$, y la validación out-of-sample mantiene el error en 1.32 puntos porcentuales. La forma en S característica de la curva de adopción se identifica claramente en los datos.
2. Los parámetros ajustados son razonables y tienen interpretación clara: $p = 0.0445$ (influencia externa moderada-alta), $q = 0.315$ (boca a boca activo), $m = 19.6\%$ (techo de mercado cercano al pico histórico de C++).
3. La predicción puntual del cruce Rust > C++ en 2028 es un resultado falsable y verificable en el corto plazo, lo que da al modelo valor predictivo concreto.

11.2 Conclusiones del proceso

1. La disponibilidad de datos públicos de calidad (Stack Overflow Survey, TIOBE, Google Trends) es lo que permite que un análisis riguroso de este tipo sea posible sin recursos institucionales. La elección del tema fue acertada en este sentido.
2. El uso combinado de herramientas de IA y validación manual permite acelerar el trabajo intelectual sin sacrificar rigor, siempre y cuando se ejerza criterio profesional sobre cada output.
3. La validación out-of-sample (corte temporal) es una técnica simple pero potente para distinguir un modelo que describe de uno que predice. Su aplicación cambia cualitativamente la confianza que se le puede tener al modelo.

11.3 Reflexión personal

Lo más desafiante del trabajo fue mantener el equilibrio entre dos objetivos en tensión: por un lado, la simplicidad del modelo (un modelo Bass con tres parámetros es deliberadamente simple); por otro, la riqueza explicativa que se espera de un análisis universitario. La salida fue agregar variantes (shock regulatorio, abandono, conservador) que mantienen la simplicidad pero permiten explorar la sensibilidad del fenómeno.

Si tuviera más tiempo, me gustaría implementar el modelo multi-tecnología que incluye competencia entre Rust, C++ y Zig. Esto requeriría datos adicionales y un sistema de ecuaciones acopladas, pero produciría un análisis estructuralmente más completo del fenómeno de "reemplazo" tecnológico, que es lo que la pregunta de investigación pedía estrictamente.

En términos de aprendizaje, el ejercicio me dejó dos lecciones aplicables a cualquier futuro proyecto de modelado: primero, que la validación importa más que el ajuste (un modelo con alto R^2 pero mala validación no sirve); segundo, que la reflexión crítica sobre las limitaciones es parte constitutiva del análisis y no un agregado opcional al final.

12. Fuentes y referencias

12.1 Fuentes de datos

- Stack Overflow Annual Developer Survey 2016 a 2025. Disponible en: <https://survey.stackoverflow.co/>
- TIOBE Programming Community Index. Disponible en: <https://www.tiobe.com/tiobe-index/>
- Google Trends — Comparativa Rust vs C++ (Topics Programming language), enero 2015 a junio 2026. Disponible en: <https://trends.google.com/trends/>

- Rust Blog — State of Rust Survey 2025. Disponible en:
<https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/>

12.2 Reportes de prensa especializada

- TechRepublic — TIOBE Programming Language Index News, julio 2024. Sobre el salto de Rust a posición 13 tras el informe del White House. Disponible en:
<https://www.techrepublic.com/article/tiobe-index-july-2024/>
- InfoWorld — Rust leaps forward in language popularity index, julio 2024. Disponible en:
<https://www.infoworld.com/article/2514539/>
- developer-tech.com — Rust enters TIOBE top 20 for the first time. Disponible en:
<https://www.developer-tech.com/news/tiobe-index-rust-top-20-first-time-c-extends-lead/>
- ZenRows — Is Rust Still Surging in 2026: Usage and Ecosystem Insights. Disponible en:
<https://www.zenrows.com/blog/rust-popularity>
- Yalantis — Rust Programming Language Adoption and Usage Trends. Disponible en:
<https://yalantis.com/blog/rust-market-overview/>

12.3 Bibliografía teórica

- Apunte Clase 1, asignatura Modelos y Simulación. Introducción a los modelos SIR y de difusión.
- Bass, F. M. (1969). A new product growth for model consumer durables. *Management Science*, 15(5), 215-227.
- Kermack, W. O., & McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society A*, 115(772), 700-721.

Anexo A — Prompts completos utilizados con IA

Esta sección documenta los prompts representativos utilizados con la herramienta de IA Claude (Anthropic) durante el desarrollo del TFI. Para cada prompt se indica la etapa del trabajo en la que se utilizó y una nota breve sobre el resultado obtenido y su validación posterior.

Prompt 1 — Etapa 1: elección del tema

"Estoy cursando Modelos y Simulación de Ingeniería en Informática. Tengo que hacer un trabajo final integrador con problemática real, recolección de datos digitales, formulación de modelo, simulación con escenarios y documentación. Dame opciones de tema que combinen un modelo de la materia (SIR, colas,

inventario, decisiones, juegos), datos digitales accesibles y relevancia para mi carrera."

Resultado: la IA propuso varias opciones agrupadas por modelo. Se eligió la opción de difusión tecnológica con datos de Stack Overflow y Google Trends.

Prompt 2 — Etapa 2: validación de series históricas

"Compilá las cifras de adopción de Rust según Stack Overflow Developer Survey 2016 a 2025. Para cada año dame: porcentaje de respondentes que usaron Rust, porcentaje que lo aman/admiran, y porcentaje que usaron C++ como referencia. Citá la fuente concreta de cada cifra y marcá los datos con baja confianza."

Resultado: se obtuvo la tabla base del dataset anual. Validación posterior contra los sitios oficiales de Stack Overflow para confirmar al menos los últimos tres años. Los datos 2016 y 2017 se marcaron como de menor confianza por la baja base estadística de Rust en esos años.

Prompt 3 — Etapa 3: limpieza del CSV de Google Trends

"Tengo este CSV de Google Trends descargado, con la comparación mensual de Rust vs C++ desde 2015. ¿Los valores tienen sentido? ¿Hay datos sospechosos que deba limpiar antes de pasar a modelar?"

Resultado: la IA identificó tres puntos críticos: el junio 2026 incompleto, el pico de normalización de marzo 2022, y la correlación entre los saltos de la serie y eventos exógenos identificables (Linux 6.1, White House, CRA EU). Las tres observaciones se validaron y quedaron documentadas en el dataset y en este informe.

Prompt 4 — Etapa 4: implementación de la simulación

"Implementá el modelo Bass de difusión en Python usando scipy. Necesito: forma analítica para ajuste rápido, forma numérica con odeint para escenarios con parámetros variables en el tiempo, ajuste por mínimos cuadrados con cotas, cálculo del R^2 y RMSE, y validación train/test temporal. Documentá las decisiones de diseño."

Resultado: se obtuvo el esqueleto del script de simulación. Hubo que ajustar las cotas iniciales porque la primera ejecución llevaba el mercado potencial al límite superior; esto se resolvió manualmente revisando la documentación de `scipy.optimize.curve_fit` y proponiendo mejores valores iniciales basados en literatura clásica de Bass.

Prompt 5 — Etapa 4: diseño de escenarios

"Para mi modelo Bass ajustado a la adopción de Rust, necesito proponer escenarios contrafácticos que respeten el principio de la consigna: variar una sola variable a la vez. Sugerime tres o cuatro escenarios que sean ingenierilmente significativos y estén bien fundamentados en la realidad observada."

Resultado: se obtuvieron los cuatro escenarios implementados en el script (base, shock regulatorio, conservador, SIR con abandono). Cada uno tiene justificación observacional explícita.

Prompt 6 — Etapa 5: estructura del documento

"Armá la estructura completa del informe del TFI siguiendo los siete puntos del producto final que pide la consigna: datos del estudiante, presentación del problema, tablas de datos, descripción del modelo y simulación, prompts utilizados, capturas y gráficos, informe final con conclusiones y reflexión crítica."

Resultado: estructura del documento que estás leyendo. Validación: chequear que todas las secciones obligatorias de la consigna están cubiertas.

Anexo B — Código fuente de la simulación

A continuación se incluyen los fragmentos clave del archivo `simulacion_bass.py`. El código completo (270 líneas aproximadamente) se entrega como archivo adjunto en el portafolio digital.

B.1 Carga de datos y definición del modelo

```
import numpy as np
import pandas as pd
from scipy.integrate import odeint
from scipy.optimize import curve_fit

df = pd.read_csv('datos_para_simulacion.csv')
t_data = df['t'].values.astype(float)
N_data = df['rust_pct'].values / 100.0
C_data = df['cpp_pct'].values / 100.0

def bass_analytical(t, p, q, m):
    """Solución analítica del modelo Bass."""
    expterm = np.exp(-(p + q) * t)
    F = (1.0 - expterm) / (1.0 + (q / p) * expterm)
    return m * F
```

B.2 Ajuste de parámetros

```
p0 = [0.003, 0.4, 0.30]
bounds = ([1e-5, 0.05, 0.10], [0.2, 2.5, 1.0])
popt, pcov = curve_fit(bass_analytical, t_data, N_data,
                       p0=p0, bounds=bounds, maxfev=10000)
p_fit, q_fit, m_fit = popt

# Bondad del ajuste
N_pred = bass_analytical(t_data, p_fit, q_fit, m_fit)
ss_res = np.sum((N_data - N_pred) ** 2)
ss_tot = np.sum((N_data - np.mean(N_data)) ** 2)
r2 = 1 - ss_res / ss_tot
rmse = np.sqrt(np.mean((N_data - N_pred) ** 2)) * 100
```

B.3 Modelo numérico para escenarios con shock

```
def bass_ode(N, t, p_func, q, m):
    p = p_func(t)
    return (p + q * N / m) * (m - N)

def p_shock(t):
    """Coeficiente de innovación que se duplica a partir de 2024."""
    return p_fit if t < 8 else p_fit * 2.0

N0 = bass_analytical(np.array([0.001]), p_fit, q_fit, m_fit)[0]
t_proj = np.linspace(0, 14, 281)
```



```
sol = odeint(bass_ode, N0, t_proj, args=(p_shock, q_fit, m_fit))
N_shock = sol.flatten()
```

B.4 Variante SIR con abandono

```
def bass_con_abandono(state, t, p, q, m, alpha):
    N, A = state # N = acumulados, A = abandonadores
    activos = N - A
    dN = (p + q * activos / m) * (m - N)
    dA = alpha * activos
    return [dN, dA]

alpha = 0.05 # 5% de abandono anual
sol_aband = odeint(bass_con_abandono, [N0, 0], t_proj,
                  args=(p_fit, q_fit, m_fit, alpha))
N_aband_activos = sol_aband[:, 0] - sol_aband[:, 1]
```